EC268 Progress Report: GGM Tree Construction

Group 2
Student: Anjana Manoj A69041661, Nikhita Neelakanta A69041742; Mark Sui A18080143

Github link: https://github.com/marksui/ECE268-GGM-Tree-Construction

Overview

Our final project is to build a pseudorandom function (PRF) tree based on the Goldreich–Goldwasser–Micali (GGM) construction. Our Goal is to build a tree structure where each parent seed expands into two child seeds, and our PRF options are: Keccak-f1600 and Spongent skeleton. We assign the project into three parts and roles as below:

- A - Anjana: Keccak-f1600 skeleton: permutation function + test vectors passing
- B - Nikhita: Spongent skeleton: sponge permutation + test vectors passing
- C - Mark: GGM tree scaffold: data structure, seed expansion interface

All members: progress report, presentation preparation, final experiments and final report.

Our Progress:

- Part A
  - Created the initial Keccak-f1600 skeleton architecture.
  - Implemented the sequential Keccak-f1600 algorithm in C++.
  - Successfully parallelized the implementation using CUDA on the UCSD DSMLP cluster.
  - Verified the structural correctness of the parallelized Keccak kernel.
  - Preparing for exhaustive testing of the CUDA Keccak implementation using varying test vectors.
  - Cross-verification of edge cases and different input sizes between the parallelized (GPU) and un-parallelized (CPU) environments to guarantee absolute mathematical equivalence is pending.

- Part B

  Started implementing the Spongent-128/128/8 skeleton (136-bit state, 1-byte rate, 70 rounds). So far the core permutation is done:

  - 7-bit LFSR for round constant generation (polynomial x^7+x+1, init 0x7E)
  - PRESENT S-box applied across all 34 nibbles of the state
  - Bit permutation pLayer: p(i) = (34i) mod 135, verified bijective via unit test

- Full 70-round permutation combining the above
- Sponge hash and domain-separated PRG expand for left/right child seeds

Basic unit tests are passing (determinism, domain separation, GGM tree integration at depth 4 and 8). GPU kernel work has started but is not yet complete.

Part C

- Planned the GGM tree scaffold.
- Start with basic tree implementation.

Challenges and Issues Encountered

Technical Difficulties:

Since our work is divided into separate parts, the GGM tree implementation depends on the completion of Part A or Part B. The tree scaffold can be designed first, might pose integration issues, but full integration and testing will need to wait until at least one backend of Keccak-f1600 or Spongent is ready.

Unexpected Experimental Behavior

We expected a linear speedup from parallelization but initially saw unexpected performance degradation. We discovered this was because the 1600-bit Keccak state was spilling from fast registers into slow, uncoalesced global/local memory, requiring aggressive #pragma unroll directives to fix.

Our backend implementation is still in development, the GGM tree hasn't integrated with Keccak-f160/Spongnet yet, so there is no unexpected experimental behavior.

Infrastructure or Debugging Issues:

The cluster automatically kills Jupyter DataHub sessions and terminal pods if they run over their time limit or remain idle. This can interrupt long-running exhaustive test suites or benchmarking scripts, requiring us to implement intermediate checkpointing.

Scope Changes or Limitations:

Large trees may expand the memory, we may have to test with smaller depths.

Timeline:

Date Range	Milestone
May 14-21	Complete Progress report and assign roles for Keccak skeleton, Spongent skeleton, GGM tree scaffold.

May 22-26	Integrate the first backend with the GGM tree scaffold and validate CPU reference expansion.
May 27-31	Integrate the second backend, compare backend outputs structurally.
Jun. 1-4	Prepare the in-class presentation.
Jun 5-12	Complete final benchmarks, record the presentation, and finish the final report.